



МИНОБРНАУКИ РОССИИ

**федеральное государственное бюджетное образовательное учреждение
высшего образования**

**«Московский государственный технологический университет «СТАНКИН»
(ФГБОУ ВО «МГТУ «СТАНКИН»)**

Основная образовательная программа 09.03.02

«Информационные системы и технологии»

**Отчет по дисциплине «Алгоритмы и структуры обработки данных»
по лабораторной работе №4**

Тема: «Хеш-таблицы и хеширование»

Проверил
к.т.н. доцент

Ковалев И.А.

подпись

Выполнила
студентка группы ИДБ-25-06

Якунина В.С.

подпись

Москва, 2026

Оглавление

Введение.....	3
Тексты программ.....	4
Задачи с Leetcode.....	4
Two sum.....	4
Valid anagram.....	5
Ransom note.....	6
Top k frequency element.....	7
First missing positive.....	8
Тесты задач с Leetcode.....	9
Хеш-таблица с открытой адресацией.....	10
Заключение.....	11

Введение

Изучить хеш-таблицы и хеширование и научиться применять их на практике.

Тексты программ

Задачи с Leetcode

Two sum

Функция, ищущая в массиве 2 числа, которые дают определенную сумму и возвращающая их, представлена ниже:

```
std::vector<int> twoSum(std::vector<int>& nums, int target) {
    std::unordered_map<int, int> hash;
    for (int i = 0; i < nums.size(); i++) {
        hash[nums[i]] = i;
    }

    int temp;
    for (int i = 0; i < nums.size(); i++) {
        temp = target - nums[i];
        if (hash.find(temp) != hash.end() && hash[temp] != i) {
            return {i, hash[temp]};
        }
    }
    return {};
}
```

Valid anagram

Функция, проверяющая, анаграмма ли введенная строка, представлена ниже:

```
bool isAnagram(std::string s, std::string t) {
    if (s.size() != t.size()) {
        return false;
    }

    std::unordered_map<char, int> hash;
    for (int i = 0; i < s.size(); i++) {
        hash[s[i]] = i;
    }

    for (char c : s) {
        hash[c]++;
    }

    for (char c : t) {
        hash[c]--;
        if (hash[c] < 0) {
            return false;
        }
    }
    return true;
}
```

Ransom note

Функция, проверяющая, хватает ли символов второй строки для создания первой, представлена ниже:

```
bool canConstruct(string ransomNote, string magazine) {
    unordered_map<int, char> hash;
    for (char simvol : magazine) {
        hash[simvol]++;
    }

    for (char simvol : ransomNote) {
        if (hash.find(simvol) != hash.end() && hash[simvol] > 0) {
            hash[simvol]--;
        }
        else {
            return false;
        }
    }
    return true;
}
```

Top k frequency element

Функция, ведущая поиск k чисел, больше всех встречающихся в массиве, представлена ниже:

```
vector<int> topKFrequent(vector<int>& nums, int k) {
    int n = nums.size();

    unordered_map<int, int> hash;

    // подсчет элементов
    for (int num : nums) {
        hash[num]++;
    }

    // пишем по корзинкам
    vector<vector<int>> buckets(nums.size() + 1);
    for (auto& pair : hash) {
        int num = pair.first;
        int count = pair.second;
        buckets[count].push_back(num);
    }

    vector<int> result;
    int temp = 0;
    for (int i = n; i >= 0; i--) {
        for (int j = 0; j < buckets[i].size(); j++) {
            result.push_back(buckets[i][j]);
            temp++;
            if (temp == k) {
                return result;
            }
        }
        if (temp == k) {
            return result;
        }
    }

    return result;
}
```

First missing positive

Функция, ищущая наименьшее положительное число, отсутствующее в массиве, представлена ниже:

```
int firstMissingPositive(vector<int>& nums) {
    unordered_map<int, int> hash;
    for (int i = 0; i < nums.size(); i++) {
        hash[nums[i]] = i;
    }

    int res = 1;
    while(1) {
        if (hash.find(res) != hash.end()) {
            res++;
        }
        else {
            return res;
        }
    }

    return 0;
}
```

Тесты задач с Leetcode

Тестирование задач проходит с помощью общей для всех функции main.

Результаты тестов показаны на рисунке 1.

```
Two sum
Input:
{2 7 11 15 }, target = 9
Output:
0 1

ValidAnagram
Input:
anagram
nagaram
Output:
1

RansomNote
Input:
aa
aab
Output:
1

TopKFrequenceElement
Input:
1 1 1 2 2 3
k = 2
Output:
1 2

FirstMissingPositive
Input:
3 4 -1 1
Output:
2
```

Рисунок 1. Тесты

Хеш-таблица с открытой адресацией

В ходе выполнения лабораторной работы понадобилась программа, работающая с хеш-таблицей с открытой адресацией:

```
#include <iostream>
#include <vector>
#include <string>

enum Status {
    FREE,
    BUSY,
    DELETED
};

struct HashNode {
    std::string key;
    int value;
    Status status;

    HashNode() : key(""), value(0), status(Status::FREE) {}
};

class HashTableOpenAddressing {
private:
    std::vector<HashNode> table;
    int capacity;
    int busy;

    int hash(std::string& key) {
        int hash = 0;
        for (char c : key) {
            hash = hash * 31 + c;
        }
        return hash;
    }

    int findIndex(std::string& key, bool& isFound) {
        int hashVal = hash(key);
        int start = hashVal % capacity;
        int foundDel = false;
        int firstDel = 0;
        int i = 0;

        while (i < capacity) {
            int cur = (start + i) % capacity;
            HashNode& node = table[cur];
```

```

        if (node.status == Status::FREE) {
            if (foundDel) {
                isFound = false;
                return firstDel;
            }
            isFound = false;
            return cur;
        }

        if (node.status == Status::DELETED && foundDel == false) {
            firstDel = cur;
        }

        if (node.status == Status::BUSY && node.key == key) {
            isFound = true;
            return cur;
        }

        ++i;
    }

    if (foundDel == true) {
        isFound = false;
        return firstDel;
    }

    isFound = false;
    return foundDel;
}

void rehash() {
    int oldCapacity = capacity;
    std::vector<HashNode> oldTable = std::move(table);

    capacity *= 2;
    table.clear();
    table.resize(capacity);

    size = 0;
    for (int i = 0; i < oldCapacity; ++i) {
        if (oldTable[i].status == Status::BUSY) {
            insert(oldTable[i].key, oldTable[i].value);
        }
    }
}

public:
    void insert(const std::string& key, int value) {
        bool isFound = false;
        int index = findIndex(key, isFound);

        if (index == SIZE_MAX) {

```

```

        std::cout << "Hash table full" << std::endl;
        rehash();
    }
    if (isFound) {
        table[index].value = value;
    }
    else {
        table[index].key = key;
        table[index].value = value;
        table[index].status = Status::BUSY;
        size++;
    }
}

void get(std::string& key, int& result) {
    bool isFound = false;
    int index = findIndex(key, isFound);

    if (!(index == SIZE_MAX || !isFound)) {
        std::cout << "Not value" << std::endl;
    }
    else {
        result = table[index].value;
    }
}

void remove(const std::string& key) {
    bool isFound = false;
    int index = findIndex(key, isFound);

    if ((!index == SIZE_MAX || !isFound)) {
        table[index].status = Status::DELETED;
        table[index].key = "";
        size--;
    }
}

bool search(const std::string& key) {
    bool isFound = false;
    findIndex(key, isFound);
    return isFound;
}

int getSize() {
    return size;
}
};

```

Заключение

В результате выполнения лабораторной работы мы научились использовать хеширование и применили на практике эти знания. Ссылка на [github](#).